Tecnológico de Costa Rica

Curso:

Lenguajes de Programación

Asignación:

Proyecto Programado 3

Profesor:

Allan Rodríguez Davila

Estudiantes:

Diego Castillo Fallas

Josimar Spencer Suarez

Melany Jiron Diaz

Semestre:

2

Año:

2026

2. Manual de usuario: instrucciones de compilación, ejecución y uso.

El programa fue ejecutado en node.js

Por lo que deberá buscar la carpeta del proyecto, abrir una terminal y ejecutar el siguiente comando:

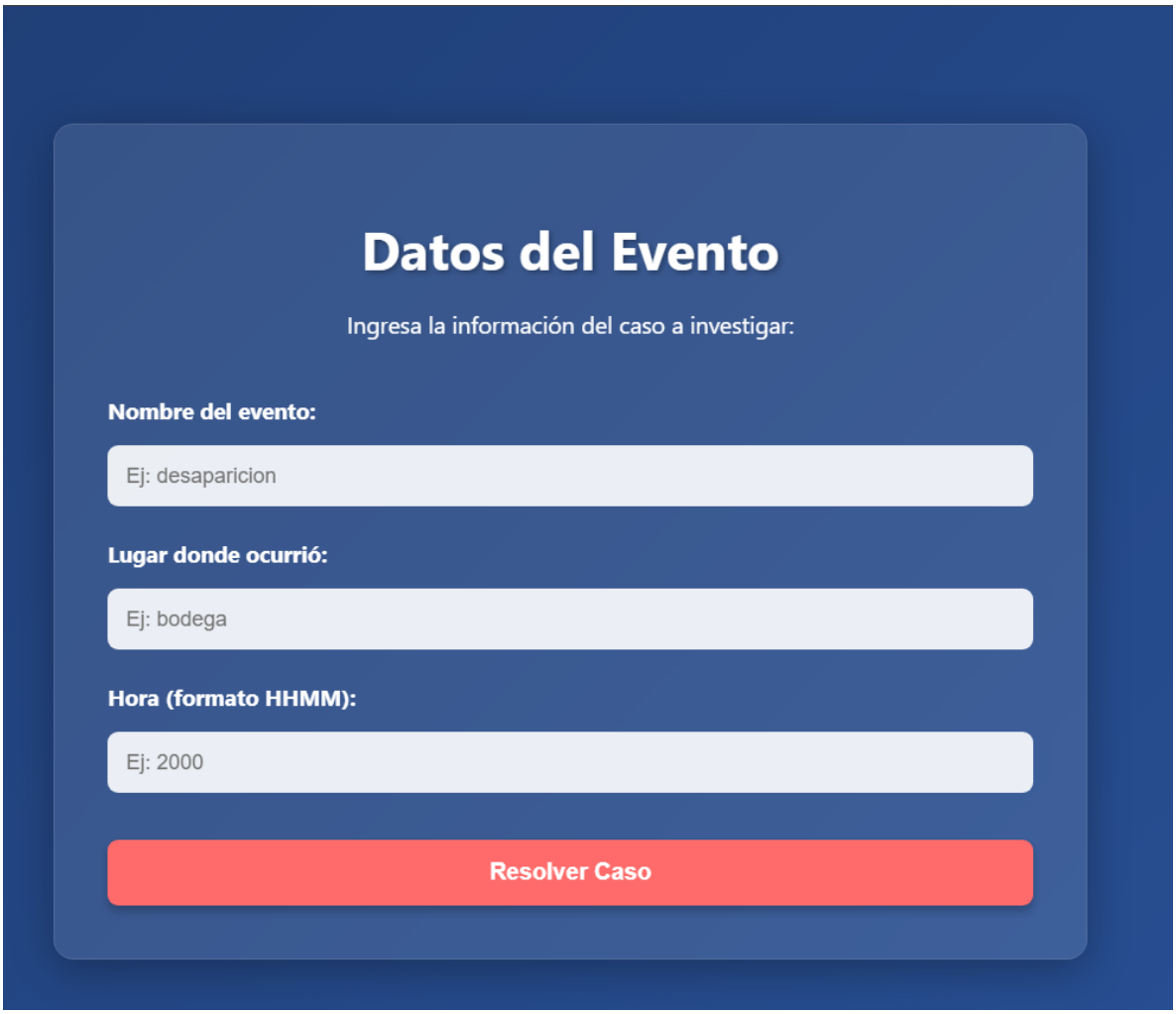
node server.js

Una vez ejecutado y pegado el link localhost en Google le saldrá la siguiente pestaña



Al cual deberá seleccionar jugar

Se le desplegara este menú:



Deberá poner un evento, el lugar donde ocurrió y hora. No podrá dejar ningún espacio en blanco y debe respetar el formato de la hora indicado en el menú.

Posteriormente se le desplegara un menú con la información ingresada (si fue valida) donde se mostrará el resultado de la investigación.



3. Pruebas de funcionalidad: incluir screenshots.



Datos del Evento

Ingresa la información del caso a investigar:

Nombre del evento:

Ej: desaparicion

Lugar donde ocurrió:

Ej: bodega

Hora (formato HHMM):

Ej: 2000

Resolver Caso



4. Descripción del problema

El proyecto Academia Detectives se pide un programa en Prolog que permita investigar un caso mediante el uso de hechos y reglas. El objetivo es que el sistema pueda analizar información sobre personas, lugares, objetos, movimientos, testimonios y evidencias para encontrar posibles responsables de un suceso.

El problema que se busca resolver es cómo relacionar toda la información de una investigación para llegar a una conclusión. En un caso pueden existir varias personas involucradas, diferentes lugares y evidencias que ayuden a entender lo ocurrido. Por eso, es necesario contar con un programa que permita revisar estos datos y encontrar relaciones entre ellos.

Para desarrollar el proyecto, se utilizó como ejemplo un caso de desaparición dentro de un museo. En este caso participan cinco personas: Juan, Mateo, Carla, Diego y Vanessa. También se registran los lugares a los que pueden ingresar, los horarios en los que estuvieron presentes, las evidencias encontradas y sus testimonios.

El programa permite consultar si una persona tenía acceso a un lugar, si estuvo presente en un momento determinado, si tiene evidencias relacionadas con el caso o si presenta contradicciones en su información.

La idea principal es que el programa no tenga guardado directamente quién es el responsable, sino que pueda encontrar posibles sospechosos mediante las reglas creadas en Prolog. De esta manera, el sistema utiliza los datos disponibles para ayudar a resolver el ca

5. Diseño del programa

5.1 Organización del programa

El proyecto fue desarrollado utilizando el lenguaje de programación Prolog, que permite trabajar con hechos y reglas para obtener respuestas.

Para mantener el código ordenado y como se solicito, se decidió separar la información del caso de las reglas que utiliza el programa.

El sistema cuenta con dos archivos principales:

- **hechos.pl:** contiene la información de las personas, lugares, objetos, accesos, movimientos, evidencias y testimonios.
- **predicados.pl:** contiene las reglas que permiten analizar la información y obtener los resultados de la investigación.

Esta organización permite cambiar los datos del caso sin tener que modificar todas las reglas del programa.

5.2 Decisiones de diseño

Separación de los archivos

Se decidió utilizar dos archivos para mantener el proyecto organizado. Uno guarda la información del caso y el otro contiene las reglas que permiten realizar las consultas.

Acceso a los lugares

Se crearon reglas para determinar si una persona puede ingresar a un lugar. El acceso puede ser directo, cuando la persona tiene permiso, o mediante un objeto, como una llave o una tarjeta.

Para el acceso mediante objetos, se estableció que una persona puede utilizarlos si estuvo en el lugar donde se encuentran.

Registro de movimientos

Se utilizaron hechos para guardar los lugares y horarios en los que estuvieron las personas. También se creó una regla que permite comprobar si alguien estuvo en un lugar dentro de un intervalo de tiempo.

Análisis de evidencias y testimonios

Se crearon reglas para relacionar las evidencias con las personas y comparar sus testimonios con los movimientos registrados.

Esto permite identificar posibles contradicciones y coartadas.

Identificación de sospechosos

El programa utiliza diferentes condiciones para determinar si una persona puede ser considerada sospechosa. Por ejemplo, si tiene evidencias relacionadas con el caso, si estuvo en el lugar del evento o si presenta contradicciones.

También se incluyeron reglas para descartar personas cuando cumplen determinadas condiciones.

Reporte de investigación

Se creó un predicado llamado resolver_caso, que reúne la información de la investigación y muestra los posibles responsables, sus evidencias, movimientos, accesos y las razones por las que fueron considerados sospechosos.

5.3 Métodos utilizados

Para desarrollar el programa se utilizaron diferentes herramientas de Prolog.

- **Unificación**
  - Permite comparar los datos de una consulta con los hechos guardados en el programa para encontrar coincidencias.
- **Backtracking**
  - Permite que Prolog busque diferentes respuestas cuando existen varias posibilidades. Por ejemplo, puede encontrar distintas personas relacionadas con un mismo lugar.
- **Creación de listas**
  - Se utilizaron instrucciones como findall y setof para reunir información sobre movimientos, accesos, evidencias y posibles responsables.
- **Comparación de datos**
  - Se utilizaron operadores para comparar horarios, lugares y condiciones. Esto permite determinar si una persona estuvo presente durante cierto tiempo o si su información presenta diferencias.

- **Reglas de investigación**
    - Se crearon diferentes predicados para analizar evidencias, coartadas, contradicciones y posibles sospechosos.
    - Estas reglas trabajan con la información guardada en hechos.pl y permiten obtener conclusiones sin tener que indicar directamente quién es el responsable.

5.4 Funcionamiento general

El programa primero carga los hechos del caso desde el archivo hechos.pl.

Después, utiliza las reglas de predicados.pl para analizar la información. Según la consulta realizada, puede mostrar los movimientos de una persona, sus accesos, las evidencias relacionadas y si presenta una posible coartada o contradicción.

Finalmente, mediante el predicado resolver_caso, el sistema reúne los resultados y presenta un reporte con los posibles responsables y las razones que apoyan esa conclusión.

Objetivo	¿Se logró?	Explicación
Crear una base de conocimiento en Prolog.	Sí	Se creó el archivo hechos.pl con la información de las personas, lugares, objetos, movimientos, evidencias y testimonios.
Implementar reglas para consultar accesos.	Sí	El programa permite comprobar si una persona tiene acceso directo a un lugar o puede ingresar mediante un objeto.
Consultar movimientos y horarios.	Sí	Se pueden consultar los lugares donde estuvo una persona y comprobar su presencia dentro de un intervalo de tiempo.
Relacionar evidencias con personas.	Sí	El sistema permite encontrar las evidencias relacionadas con cada persona involucrada.
Identificar coartadas y contradicciones.	Sí	Se pueden comparar los testimonios con los movimientos registrados para encontrar posibles

		coartadas y contradicciones.
Identificar posibles sospechosos.	Sí	El programa analiza las evidencias, los accesos y los movimientos para identificar posibles responsables.
Descartar personas.	Sí	Se utilizan reglas para descartar personas cuando cumplen las condiciones establecidas.
Generar explicaciones.	Sí	El sistema muestra razones que permiten comprender por qué una persona puede ser sospechosa o descartada.
Resolver el caso.	Sí	Se genera un reporte con los posibles responsables, las evidencias y la información que respalda los resultados.
Implementar una interfaz gráfica.	Sí	Se cuenta con una interfaz que permite utilizar las funciones del sistema y consultar los resultados.
Aplicar unificación y backtracking.	Sí	Se utilizaron estos mecanismos de Prolog para relacionar la información y encontrar diferentes soluciones.
Comprobar el funcionamiento del proyecto.	Sí	Según las pruebas realizadas por el equipo, el programa funciona correctamente.

7. Análisis de predicados

Se describe cómo funciona cada uno, por qué se utilizaron determinadas condiciones y de qué manera ayudan a resolver el caso.

7.1 Predicado tiene_contradiccion/1

Objetivo

Este predicado ayuda a identificar si una persona presenta una contradicción entre su testimonio y los movimientos registrados en la base de conocimiento.

Implementación

```
tiene_contradiccion(Persona) :-
  testimonio(Persona, Lugar, Hora),
  \+ estuvo(Persona, Lugar, Hora).
```

Decisiones tomadas y justificación

Para implementar este predicado comparamos dos tipos de información: los testimonios y los registros de movimientos.

Primero utilizamos testimonio(Persona, Lugar, Hora) para obtener el lugar y la hora que una persona indicó en su declaración.

Después se usó \+ estuvo(Persona, Lugar, Hora) para ver si no existe un registro que confirme que esa persona estuvo en el lugar y la hora mencionados.

Se eligió el operador \+ porque ayuda comprobar que una condición no puede demostrarse con los hechos disponibles.

La decisión de comparar el lugar y la hora exactos se implementó para realizar una consulta sencilla y directa, utilizando los datos que ya se encuentran en el archivo hechos.pl.

¿Por qué se implementó de esta manera?

Se implemento así ya que la idea era que el sistema detectara posibles contradicciones automáticamente, sin necesidad de agregar un hecho que indique directamente que una persona está mintiendo.

Digamos, si Mateo declara que estuvo en la entrada a las 19:55, pero no existe un registro que confirme esa ubicación y hora, el predicado considera que existe una contradicción.

Importancia dentro del programa

Este predicado es importante porque sus resultados se utilizan posteriormente para identificar posibles sospechosos, ver si una persona puede ser descartada y generar explicaciones.

7.2 Predicado posible_sospechoso/1

Objetivo

Este predicado permite ver si una persona cumple alguna de las condiciones establecidas para ser considerada posible sospechosa de un evento.

Implementación

```
posible_sospechoso(Persona) :-
  persona(Persona),
  evento(_, LugarEvento, _),
  evidencia(Evidencia, LugarEvento, _),
  corresponde(Evidencia, Persona).


posible_sospechoso(Persona) :-
  persona(Persona),
  evento(_, LugarEvento, HoraEvento),
  tiene_acceso(Persona, LugarEvento),
  estuvo(Persona, LugarEvento, HoraEvento).


posible_sospechoso(Persona) :-
  persona(Persona),
  tiene_contradiccion(Persona),
  evento(_, LugarEvento, _),
  (tiene_acceso(Persona, LugarEvento) ; corresponde(_, Persona)).
```

Decisiones tomadas y justificación

Se decidió implementar este predicado mediante tres reglas diferentes, ya que una persona puede ser considerada sospechosa por distintas razones.

Primera regla: relación con una evidencia

La primera regla ve si una persona está relacionada con una evidencia encontrada en el lugar donde ocurrió el evento.

Para eso se obtiene el lugar del evento mediante evento/3, se buscan las evidencias encontradas en ese lugar mediante evidencia/3 y finalmente se utiliza corresponde/2 para comprobar a qué persona pertenece la evidencia.

Se eligió esto porque una evidencia relacionada con una persona puede ser importante para la investigación.

Segunda regla: acceso y presencia en el evento

La segunda regla es para ver si una persona tenía acceso al lugar del evento y si existe un registro que indique que estuvo allí a la misma hora en que ocurrió.

Se decidió usar ambas condiciones porque tener acceso a un lugar no significa necesariamente que una persona haya estado presente durante el evento.

Por eso, el programa pide que se cumplan las dos condiciones para considerar a una persona sospechosa mediante esta regla.

Tercera regla: contradicciones y relación con el caso

La tercera regla ve si una persona presenta contradicciones y, además, tiene acceso al lugar del evento o está relacionada con alguna evidencia.

Se usa el operador ; para indicar que basta con cumplir una de estas dos últimas condiciones.

Esta decisión permite considerar personas que tienen información contradictoria y que también tienen alguna relación con la investigación.

¿Por qué se implementó de esta manera?

Se decidió usar estas reglas porque no todas las personas sospechosas presentan las mismas características.

Algunas pueden estar relacionadas con evidencias, otras pueden haber estado presentes durante el evento y otras pueden presentar contradicciones.

Prolog permite utilizar varias reglas con el mismo nombre para encontrar diferentes soluciones mediante backtracking.

Importancia dentro del programa

Este predicado es uno en lo que se basa el sistema, ya que permite identificar a las personas que deben ser consideradas durante la investigación.

Además, sus resultados son utilizados por posibles_responsables/1.

7.3 Predicado descartar/1

Objetivo

Este predicado ayuda a determinar si una persona puede ser descartada como posible responsable del evento.

Implementación

```
descartar(Persona) :-
  persona(Persona),
  tiene_coartada(Persona),
  \+ tiene_contradiccion(Persona),
  \+ evidencia_en_evento(Persona).
```

Decisiones tomadas y justificación

Para implementar este predicado se decidió utilizar tres condiciones principales.

Primera condición: tener una coartada

Se utiliza tiene_coartada(Persona) para comprobar si la persona cuenta con información que deje ubicarla en un lugar diferente al del evento.

Se eligió esta condición porque una coartada puede ayudar a demostrar que una persona no estuvo relacionada directamente con el suceso.

Segunda condición: no presentar contradicciones

Se utiliza \+ tiene_contradiccion(Persona) para comprobar que no se haya detectado una contradicción según las reglas del sistema.

Esta condición es para ver si una persona con información contradictoria requiere mayor revisión antes de ser descartada.

Tercera condición: no tener evidencias relacionadas con el lugar del evento

Se utiliza \+ evidencia_en_evento(Persona) para comprobar que no existan evidencias relacionadas con esa persona en el lugar donde ocurrió el evento.

Se eligió esta condición para evitar descartar personas que todavía presentan una relación importante con el caso.

¿Por qué se implementó de esta manera?

Se decidió exigir que las tres condiciones se cumplan al mismo tiempo.

Esto significa que una persona no será descartada únicamente por tener una coartada. También debe cumplir las otras dos condiciones.

Esta decisión busca que el descarte se realice utilizando diferentes datos de la investigación.

Además, se decidió crear el predicado auxiliar evidencia_en_evento/1 para separar la búsqueda de evidencias de la regla principal de descarte.

Importancia dentro del programa

Este predicado nos permite reducir la lista de posibles responsables y evitar que se incluyan personas que cumplen las condiciones establecidas para ser descartadas.

7.4 Predicado posibles_responsables/1

Objetivo

Este predicado nos permite obtener una lista de las personas que cumplen las condiciones para ser consideradas posibles responsables y que no han sido descartadas.

Implementación

```
posibles_responsables(Lista) :-
  setof(Persona,
    (persona(Persona),
     posible_sospechoso(Persona),
     \+ descartar(Persona)),
    Lista).
```

Decisiones tomadas y justificación

Se decidió utilizar setof/3 para reunir en una sola lista a las personas que cumplen las condiciones de la investigación.

Primero, persona(Persona) permite recorrer las personas registradas en la base de conocimiento.

Después, posible_sospechoso(Persona) comprueba si cada persona cumple alguna de las condiciones para ser considerada sospechosa.

Por último el \+ descartar(Persona) permite excluir a las personas que cumplen las condiciones de descarte.

Se eligió setof/3 porque permite obtener una lista ordenada y sin elementos repetidos.

Esto es importante porque una misma persona podría cumplir varias reglas de posible_sospechoso/1, pero solamente debe aparecer una vez en el resultado final.

¿Por qué se implementó de esta manera?

Se buscó que el programa pudiera analizar a todas las personas registradas y construir automáticamente una lista de posibles responsables.

En sí, no es necesario indicar manualmente quiénes son los sospechosos.

También se decidió utilizar los predicados posible_sospechoso/1 y descartar/1 para aprovechar las reglas que ya habían sido implementadas.

Importancia dentro del programa

Este predicado nos permite reunir los resultados de la investigación y proporciona la lista que posteriormente utiliza resolver_caso/0 para generar el reporte final.

7.5 Predicado explicaciones/2

Objetivo

Este predicado nos ayuda a generar diferentes explicaciones sobre la relación de una persona con el caso investigado.

Implementación

Se decidió crear varias reglas para explicaciones/2, cada una relacionada con una situación diferente.

Las explicaciones consideradas en el código son:

1. La persona tenía acceso al lugar del evento.

2. La persona estuvo registrada en el lugar del evento según una consulta de intervalo.
3. Existe una evidencia relacionada con la persona.
4. La persona presenta contradicciones.
5. La persona no tiene una coartada válida según las reglas.
6. La persona cumple las condiciones para ser descartada.

Decisiones tomadas y justificación

Se decidió separar las explicaciones en varias reglas porque una persona puede tener más de una relación con el caso.

Por ejemplo, una persona puede tener acceso al lugar del evento y también estar relacionada con una evidencia.

Si todas las explicaciones se manejaran mediante una sola condición, sería más difícil obtener diferentes razones para analizar a una misma persona.

Por eso, estas se usan varias reglas con el mismo nombre y los mismos argumentos.

También se decidió utilizar atomic_list_concat/2 en algunas reglas para construir mensajes que incluyan información obtenida de los hechos, como el lugar del evento o el nombre de una evidencia.

En otras reglas se utilizan mensajes de texto que describen directamente la condición encontrada.

¿Por qué se implementó de esta manera?

Se buscó que el sistema no se limitara a mostrar el nombre de una persona sospechosa, sino que también presentara las razones que permiten relacionarla con el caso.

Esto facilita que el usuario comprenda los resultados de la investigación.

Además, el uso de varias reglas permite aprovechar el backtracking de Prolog para obtener diferentes explicaciones cuando una persona cumple varias condiciones.

Importancia dentro del programa

Este predicado es importante porque permite justificar los resultados y mostrar qué información fue utilizada durante la investigación.

Sus explicaciones también aparecen dentro del reporte generado por resolver_caso/0.

7.6 Predicado resolver_caso/0

Objetivo

Este predicado permite realizar el proceso general de investigación y generar un reporte con los posibles responsables del evento.

Decisiones tomadas y justificación

Se decidió implementar resolver_caso/0 como el predicado principal que reúne los resultados obtenidos mediante las demás reglas del sistema.

En lugar de repetir toda la lógica de investigación, se utilizan los predicados que ya fueron desarrollados.

El proceso se organizó en diferentes etapas.

Primera etapa: obtener los datos del evento

El programa utiliza evento/3 para obtener el nombre, el lugar y la hora del suceso.

Se decidió iniciar con esta información porque permite identificar el caso que se está investigando.

Segunda etapa: obtener los posibles responsables

Se utiliza posibles_responsables/1 para obtener la lista de personas que cumplen las condiciones establecidas.

Esta decisión permite aprovechar las reglas de sospechosos y descartes sin repetirlas dentro del predicado principal.

Tercera etapa: consultar la información de cada persona

Se utiliza forall/2 para recorrer la lista de posibles responsables y mostrar la información correspondiente a cada uno.

Para cada persona se consultan las evidencias, los movimientos, los accesos, las coartadas y las contradicciones.

Se decidió mostrar estos datos porque permiten comprender las relaciones que tiene cada persona con el caso.

Cuarta etapa: mostrar las explicaciones

Se utiliza explicaciones/2 para obtener las razones relacionadas con cada posible responsable.

Esta decisión permite presentar información adicional que justifica por qué una persona aparece en el reporte.

Quinta etapa: mostrar las personas descartadas

El programa utiliza descartar/1 para encontrar las personas que cumplen las condiciones de descarte.

Se decidió incluir esta sección porque la investigación no solamente debe identificar sospechosos, sino también mostrar cuáles personas pueden ser excluidas según las reglas establecidas.

Sexta etapa: generar la conclusión

Finalmente, el programa muestra la cantidad de posibles responsables y sus nombres.

Para esto se utiliza length/2, que permite contar los elementos de la lista.

También se incluyó una condición para mostrar un mensaje cuando la lista de responsables esté vacía.

¿Por qué se implementó de esta manera?

Se buscó organizar el proceso de investigación en un solo predicado que permitiera consultar la información más importante del caso.

Esto evita que el usuario tenga que ejecutar cada consulta por separado para conocer los resultados generales.

Además, se decidió presentar la información mediante mensajes organizados para facilitar su lectura.

Importancia dentro del programa

resolver_caso/0 es el predicado encargado de reunir las diferentes partes de la investigación y presentar una conclusión basada en las reglas del sistema.

No tiene almacenado directamente el nombre del responsable, sino que obtiene los resultados a partir de la información disponible.